

浙江大学 实验报告

专业： 信息工程

姓名：

学号：

日期： 2026 年 4 月 8 日

地点： 东四-223

课程名称： 数字系统设计实验

指导老师： 屈民军，唐奕

成绩：

实验名称： 常用组合电路模块的设计和应用

实验类型： 设计性实验

同组学生姓名： 无

一、 实验目的

1. 掌握用 Verilog HDL 描述数据选择器、加法器和比较器等电路模块。
2. 了解“自顶而下”的数字设计方法，掌握系统层次结构的设计。
3. 掌握模块调用的方法，掌握参数定义和参数传递的方法。
4. 掌握 ModelSim 功能仿真的工作流程，进一步了解 Vivado 的工作流程。
5. 认识到文件管理的重要性。

二、 实验任务

1. 设计两数之差的绝对值电路：电路输入 aIn 、 bIn 为 4 位无符号二进制数，电路输出 out 为两数之差的绝对值，即 $out = |aIn - bIn|$ 。要求用多层次结构设计电路，即调用数据选择器、加法器和比较器等基本模块来设计电路。
2. 设计模式比较器 (Mode-Dependent Comparator) 电路：电路的输入为两个 8 位无符号二进制数 a 、 b 和一个模式控制信号 m ；电路的输出为 8 位无符号二进制数 y 。当 $m=0$ 时， $y=MAX(a,b)$ ；而当 $m=1$ 时，则 $y=MIN(a,b)$ 。要求用多层次结构设计电路，即调用数据选择器和比较器等基本模块来设计电路。

三、 实验原理

3.1 绝对值之差电路

3.1.1 顶层设计

两数之差的绝对值电路的总体设计：两个无符号数之差的绝对值电路可由式(3.1)计算。另外，减法可由补码加法完成，因此式(3.1)可转换为式(3.2)实现。

$$out = Max(aIn, bIn) - Min(aIn, bIn) \quad (3.1)$$

$$out = Max(aIn, bIn) + (\sim Min(aIn, bIn) + 1) \quad (3.2)$$

根据式(3.2)可画出两数之差的绝对值电路的顶层设计,如图3.1所示。图中电路主要由三部分组成:数值比较器(comp)实现输入两数的大小比较;比较结果 agb 控制两个数据选择器,数据选择器分别选出输入两数的最大值 Max 和最小值 Min;最后由全加器组成的四位加法器实现 Max 与 $(-Min)$ 的补码加法。

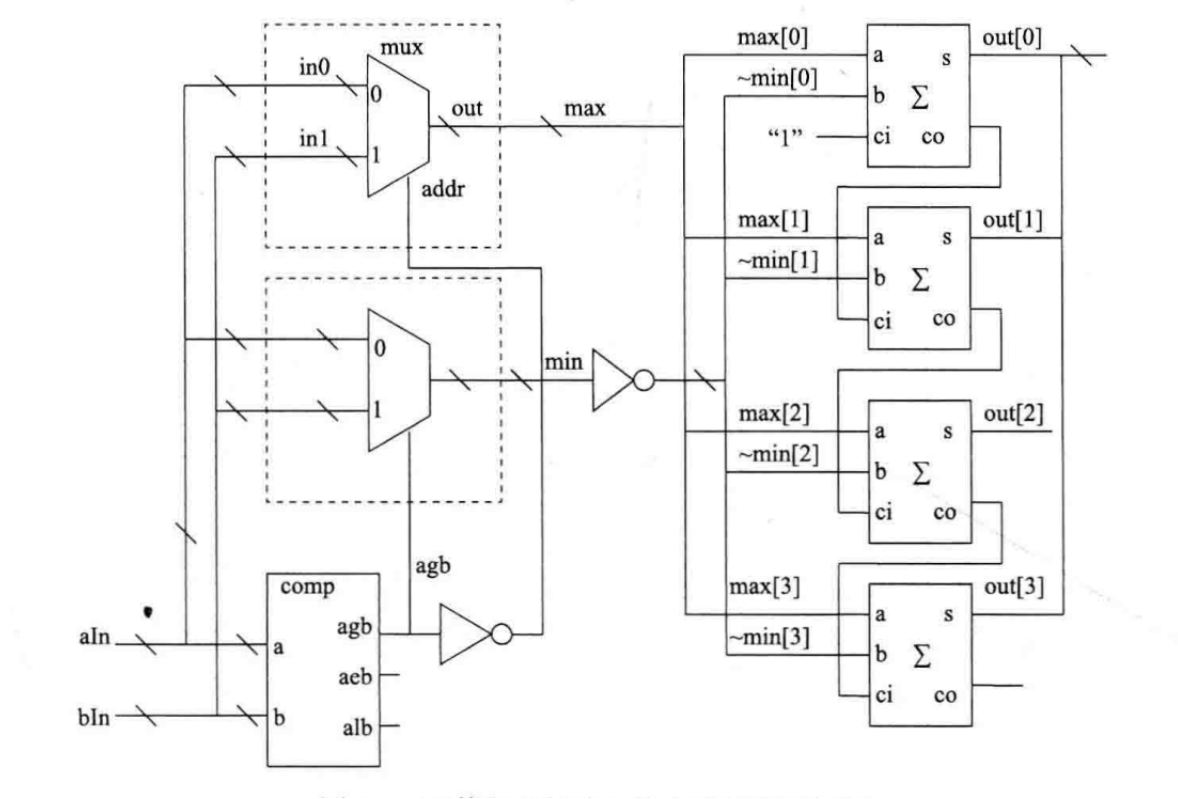


图 3.1: 绝对值之差电路的顶层设计

代码实现如下 (由书上提供样例):

Listing 1: 绝对值之差电路顶层设计

```

1 module abs_dif(aIn, bIn, out);
2     input [3:0] aIn, bIn;
3     output [3:0] out;
4
5     // 比较器实例, 在此实例描述中, 注意空脚的连接方法
6     wire agb;
7     comp #(n(4)) comp_inst(.a(aIn), .b(bIn), .agb(agb), .acb(), .alb());
8
9     // 数据选择器实例, 注意addr 信号的连接及参数的传递, 注意实例名。
10    wire [3:0] max, min;
11    mux_2to1 #(n(4)) mux1(.out(max), .in0(aIn), .in1(bIn), .addr(~agb));
12    mux_2to1 #(n(4)) mux2(.out(min), .in0(aIn), .in1(bIn), .addr(agb));
13

```

```

14 // 全加器实例, 注意信号组可拆开使用以及端口接常数的方法
15 wire [2:0] c;
16 full_adder adder0(.a(max[0]), .b(~min[0]), .s(out[0]), .ci(1'b1), .co(c[0]));
17 full_adder adder1(.a(max[1]), .b(~min[1]), .s(out[1]), .ci(c[0]), .co(c[1]));
18 full_adder adder2(.a(max[2]), .b(~min[2]), .s(out[2]), .ci(c[1]), .co(c[2]));
19 full_adder adder3(.a(max[3]), .b(~min[3]), .s(out[3]), .ci(c[2]), .co());
20
21 endmodule

```

3.1.2 比较器

比较器模块用于比较两个 n 位无符号二进制数 a 和 b 的大小关系, 输出三个标志信号: agb ($a > b$)、 aeb ($a = b$) 和 alb ($a < b$)。该模块采用参数化设计, 通过参数 n 指定位宽, 默认值为 8 位。比较器的工作原理是从最高位向最低位逐位比较: 当首次出现某一位 $a[i] > b[i]$ 时, 判定 $a > b$; 若出现 $a[i] < b[i]$, 则判定 $a < b$; 若所有位均相等, 则两数相等。实现中使用 **for** 循环遍历每一位, 并借助 **quit** 信号在得到比较结果后提前终止循环, 以提高效率。比较器的 Verilog 代码如下:

Listing 2: 比较器模块 comp

```

1 // module for comparator
2 module comp #(parameter n = 8) (
3     input  [n-1:0] a,
4     input  [n-1:0] b,
5     output reg      agb, // a > b
6     output reg      aeb, // a = b
7     output reg      alb // a < b
8 );
9     integer i;
10    reg      quit;
11
12    always @(*) begin
13        agb = 0; alb = 0; aeb = 1; // assume a = b by default
14        quit = 0; // decide whether to quit the loop
15        for (i = n - 1; i >= 0 && !quit; i = i - 1) begin
16            if (a[i] > b[i]) begin // a is greater
17                agb = 1; alb = 0; aeb = 0;
18                quit = 1;
19            end
20            else if (a[i] < b[i]) begin // b is greater
21                agb = 0; alb = 1; aeb = 0;

```

```

22         quit = 1;
23     end
24 end
25 end
26 endmodule

```

该比较器模块可被上层电路调用，在绝对值之差电路（图 3.1）中，实例化比较器以判断输入 **aIn** 与 **bIn** 的大小关系，其输出 **agb** 用于控制数据选择器选出最大值和最小值。

3.1.3 数据选择器

数据选择器模块用于根据地址信号 **addr** 从两个 n 位输入 **in0** 和 **in1** 中选择其一作为输出 **out**。该模块同样采用参数化设计，通过参数 **n** 指定位宽，默认值为 8 位。当 **addr** 为 0 时，输出等于 **in0**；当 **addr** 为 1 时，输出等于 **in1**。数据选择器的 Verilog 代码如下：

Listing 3: 二选一数据选择器模块 mux_2to1

```

1  // module for 2-to-1 multiplexer
2  module mux_2to1 #(parameter n = 8) (
3      input      [n-1:0] in0,
4      input      [n-1:0] in1,
5      input      addr,    // 0: out = in0; 1: out = in1
6      output reg [n-1:0] out
7  );
8      always @(*) begin
9          if (addr == 0) begin
10             out = in0;
11         end
12         else begin
13             out = in1;
14         end
15     end
16 endmodule

```

在绝对值之差电路（图 3.1）中，两个数据选择器实例分别用于选出最大值 **max** 和最小值 **min**。其中，第一个数据选择器的地址信号为 **agb**（比较器输出的取反），因此当 **agb**=1（即 $aIn > bIn$ ）时，地址为 0，输出 **in0**（即 **aIn**）作为最大值；反之则输出 **in1**（即 **bIn**）。第二个数据选择器的地址直接为 **agb**，用于选出最小值。

3.1.4 一位全加器

一位全加器模块用于实现两个一位二进制数 **a**、**b** 与进位输入 **ci** 的加法运算, 输出和 **s** 以及进位输出 **co**。其逻辑表达式为:

$$s = a \oplus b \oplus ci$$

$$co = (a \wedge b) \vee (a \wedge ci) \vee (b \wedge ci)$$

其中 $\oplus$ 表示异或运算, $\wedge$ 表示与运算, $\vee$ 表示或运算。该模块的行为使用组合逻辑描述, 通过过程块实现。一位全加器的 Verilog 代码如下:

Listing 4: 一位全加器模块 full_adder

```
1 // module for 1-bit full adder
2 module full_adder (
3     input a,
4     input b,
5     input ci,
6     output reg s,
7     output reg co
8 );
9     always @(*) begin
10         s = a ^ b ^ ci;
11         co = (a & b) | (a & ci) | (b & ci);
12     end
13 endmodule
```

在绝对值之差电路(图 3.1)中, 四位加法器由四个一位全加器级联构成, 用于实现最大值 **Max** 与最小值取反加一(即 **-Min** 的补码)的加法运算。其中第一个全加器的进位输入 **ci** 固定为 1, 对应补码加法中的加一操作; 后续全加器的进位输入依次连接前一级的进位输出, 构成行波进位加法器。每个全加器的 **a** 输入端连接 **Max** 的对应位, **b** 输入端连接 $\sim \text{Min}$ 的对应位(按位取反), 和输出 **s** 直接作为结果的对应位, 最高位的进位输出悬空(因结果仍为 4 位, 溢出忽略)。

3.2 模式比较器电路

模式比较器用于根据模式控制信号 **m** 输出两个 8 位无符号二进制数 **a** 和 **b** 的最大值或最小值。当 **m=0** 时, 输出最大值 **MAX(a,b)**; 当 **m=1** 时, 输出最小值 **MIN(a,b)**。

该电路采用多层次结构设计, 通过调用已实现的比较器模块 **comp** 判断 **a** 与 **b** 的大小关系, 然后利用组合逻辑根据比较结果和模式信号选出最终输出。顶层设计如图 3.2 所示。

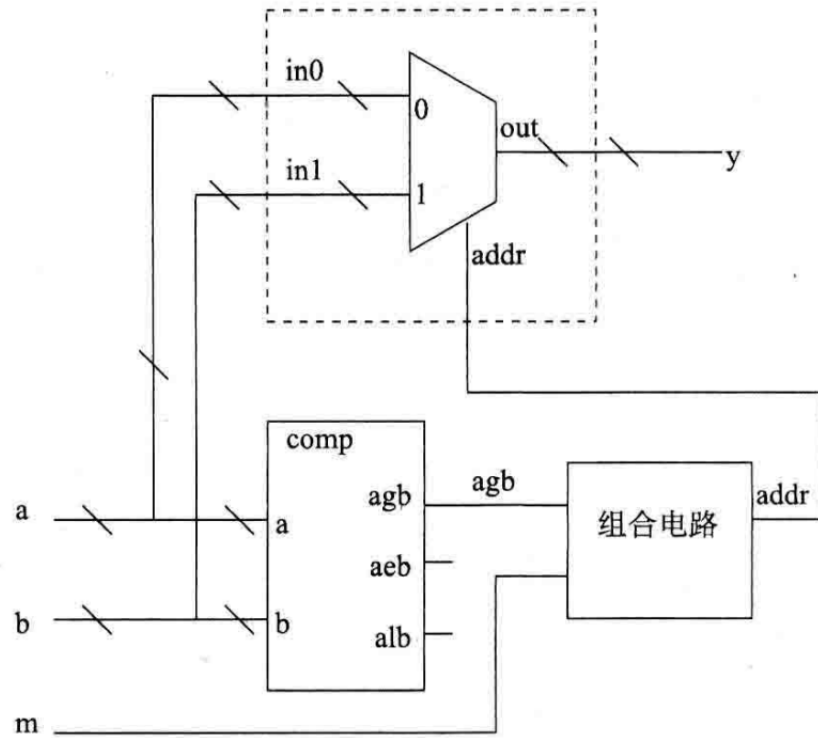


图 3.2: 模式比较器电路顶层设计

模式比较器的 Verilog 代码如下:

Listing 5: 模式比较器模块 ModeComparator

```

1 // module for mode comparator
2 module ModeComparator #(parameter n = 8)(
3     input[n-1:0]    a,
4     input[n-1:0]    b,
5     input            m, // m = 0: output maximum; m = 1: output minimum
6     output reg[n-1:0] y
7 );
8
9     wire agb; // a is greater than b
10    reg [n-1:0] min_out;
11    reg [n-1:0] max_out;
12
13    comp #(.n(n)) comp1(
14        .a(a), .b(b),
15        .agb(agb), .aeb(), .alb()
16    );
17

```

```
18     always @(*) begin
19         if (agb == 1) begin // a is greater than b
20             max_out = a;
21             min_out = b;
22         end
23         else begin // a is less than or equal to b
24             min_out = a;
25             max_out = b;
26         end
27
28         if (m == 1) y = min_out;
29         else y = max_out;
30     end
31 endmodule
```

该模块首先实例化比较器 **comp1**, 其输出 **agb** 指示 $a > b$ 是否成立。在组合逻辑块中, 根据 **agb** 确定最大值 **max_out** 和最小值 **min_out**: 若 $a > b$, 则 **max_out**=a, **min_out**=b; 否则 **max_out**=b, **min_out**=a。最后, 依据模式信号 **m** 选择输出: 当 **m**=1 时输出最小值, 否则输出最大值。

四、 实验设备

1. 装有 Vivado 和 ModelSim SE 软件的计算机
2. Nexys Video 开发板或 Baysy3 开发板

五、 实验步骤

5.1 实验一: 绝对值之差电路

1. 编写一位全加器的 Verilog HDL 代码, 并用 ModelSim 软件进行功能仿真。
2. 编写 n 位二选一数据选择器的 Verilog HDL 代码, 并用 ModelSim 软件进行功能仿真。
3. 编写 n 位比较器的 Verilog HDL 代码, 并用 ModelSim 软件进行功能仿真。
4. 对两数之差的绝对值电路进行功能仿真。
5. 建立 Vivado 工程文件, 对工程进行综合、引脚约束、实现, 并下载到开发实验板中对设计进行验证。

5.2 实验二: 模式比较器电路

编写模式比较器的 Verilog HDL 代码, 并用 ModelSim 等软件进行功能仿真。

六、 仿真分析

6.1 数据比较器仿真分析

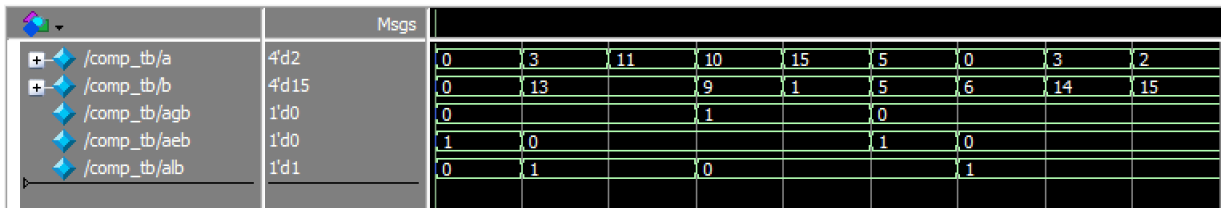


图 6.1: 数据比较器 ModelSim 波形仿真

<i>a</i> (十进制)	<i>b</i> (十进制)	<i>agb</i> (<i>a</i> > <i>b</i>)	<i>aeb</i> (<i>a</i> = <i>b</i>)	<i>alb</i> (<i>a</i> < <i>b</i>)
0	0	0	1	0
3	13	0	0	1
11	9	1	0	0
10	1	1	0	0
15	1	1	0	0
5	5	0	1	0
0	6	0	0	1
3	14	0	0	1
2	5	0	0	1

表 6.1: 4 位比较器 ModelSim 仿真波形结果

为验证比较器模块的功能正确性，使用 ModelSim 对 4 位比较器（参数 **n=4**）进行了功能仿真。仿真波形如图 6.1 所示，对应的输入输出数据记录在表 6.1 中。

从仿真结果可以看出，比较器能够正确判断两个无符号二进制数 *a* 和 *b* 的大小关系：

- 当 *a* = *b* 时，输出 **agb=0**，**aeb=1**，**alb=0**。例如表中第一行（0 与 0）和第五行（5 与 5）均满足该情况。
- 当 *a* > *b* 时，输出 **agb=1**，**aeb=0**，**alb=0**。如第三行（11 > 9）、第四行（10 > 1）等，波形中 **agb** 信号均为高电平。
- 当 *a* < *b* 时，输出 **agb=0**，**aeb=0**，**alb=1**。例如第二行（3 < 13）、第六行（0 < 6）等，波形中 **alb** 信号为高电平。

所有测试用例均符合预期，证明比较器模块的功能正确。该模块通过参数 **n** 可灵活适配不同位宽，为后续绝对值之差电路和模式比较器电路提供了可靠的大小判断依据。

6.2 数据选择器仿真分析

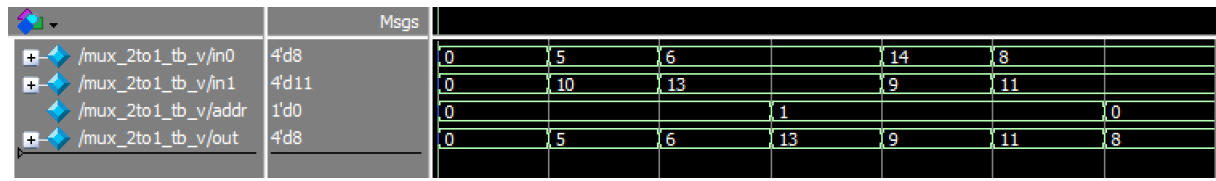


图 6.2: 数据选择器 ModelSim 波形仿真

<i>in0</i> (十进制)	<i>in1</i> (十进制)	<i>addr</i> (选择端)	<i>out</i> (输出)
0	0	0	0
5	10	0	5
6	13	0	6
6	13	1	13
14	9	1	9
8	11	1	11
8	11	0	8

表 6.2: 4 位二选一数据选择器 ModelSim 仿真波形结果

为验证二选一数据选择器模块的功能正确性，使用 ModelSim 对 4 位数据选择器（参数 **n=4**）进行了功能仿真。仿真波形如图 6.2 所示，对应的输入输出数据记录在表 6.2 中。

数据选择器根据地址信号 **addr** 从两个输入 **in0** 和 **in1** 中选择一路输出：

- 当 **addr=0** 时，输出 **out** 等于 **in0**。例如表中第二行（**in0=5**，**in1=10**，**addr=0**），输出为 5；第三行（**in0=6**，**in1=13**，**addr=0**），输出为 6；第七行（**in0=8**，**in1=11**，**addr=0**），输出为 8。
- 当 **addr=1** 时，输出 **out** 等于 **in1**。例如第四行（**in0=6**，**in1=13**，**addr=1**），输出为 13；第五行（**in0=14**，**in1=9**，**addr=1**），输出为 9；第六行（**in0=8**，**in1=11**，**addr=1**），输出为 11。

所有测试用例的输出均与预期一致，证明数据选择器模块的功能正确。该模块采用参数化设计，可通过修改参数 **n** 适配不同位宽的数据选择，为绝对值之差电路和模式比较器电路提供了灵活的数据通路选择功能。

6.3 一位全加器仿真分析

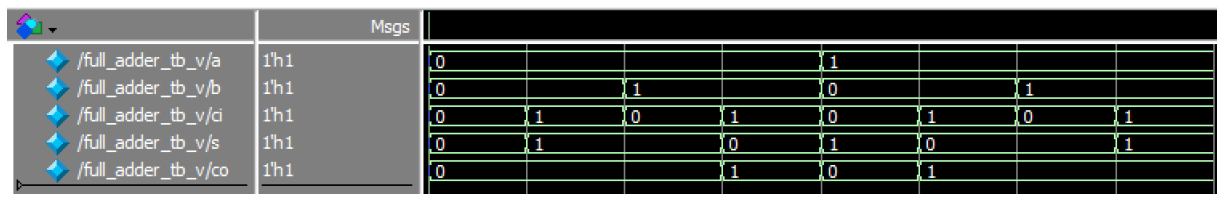


图 6.3: 一位全加器 ModelSim 波形仿真

a	b	ci	s	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

表 6.3: 一位全加器 ModelSim 仿真波形结果

为验证一位全加器模块的功能正确性，使用 ModelSim 对一位全加器进行了功能仿真。仿真波形如图 6.3 所示，对应的输入输出数据记录在表 6.3 中。

一位全加器实现两个一位二进制数 **a**、**b** 与进位输入 **ci** 的加法运算，输出和 **s** 以及进位输出 **co**，其逻辑表达式为：

$$s = a \oplus b \oplus ci, \quad co = (a \wedge b) \vee (a \wedge ci) \vee (b \wedge ci)$$

从仿真结果可以看出，全加器在所有 8 种输入组合下均能正确输出：

- 当输入中 1 的个数为奇数时，和 **s**=1；为偶数时，**s**=0。例如 **a**=0,**b**=0,**ci**=1（1 个 1），**s**=1；**a**=1,**b**=1,**ci**=1（3 个 1），**s**=1。
- 进位输出 **co** 在输入中至少有两个 1 时为 1，否则为 0。例如 **a**=0,**b**=1,**ci**=1（两个 1），**co**=1；**a**=1,**b**=1,**ci**=0（两个 1），**co**=1；仅有 **a**=1,**b**=1,**ci**=1（三个 1）时，**co**=1。

所有测试用例均符合预期，证明一位全加器模块的功能正确。该模块是构成多位加法器的基本单元，在绝对值之差电路中，四个全加器级联形成四位行波进位加法器，用于实现最大值与最小值补码的加法运算。

6.4 绝对值之差电路仿真分析

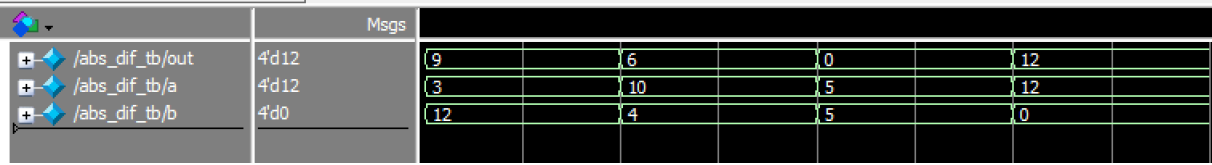


图 6.4: 绝对值之差电路 ModelSim 波形仿真

```
VSIM 21> run -all
# | 3 - 12 |= 9
# | 10 - 4 |= 6
# | 5 - 5 |= 0
# | 12 - 0 |= 12
# ** Note: $stop : C:/Users/57195/Desktop/SB2JU/DigitalSystemDesignEx/lab5/src/abs_dif_tb.v(18)
# Time: 80 ns Iteration: 0 Instance: /abs_dif_tb
```

图 6.5: 绝对值之差电路 ModelSim 控制台输出

<i>a</i> (十进制)	<i>b</i> (十进制)	<i>out</i> (输出, $ a - b $)
3	12	9
10	4	6
5	5	0
12	0	12

表 6.4: 两数之差的绝对值电路 ModelSim 仿真波形结果

为验证两数之差的绝对值电路的功能正确性，使用 ModelSim 对电路进行了功能仿真。仿真波形如图 6.4 所示，控制台输出如图 6.5 所示，对应的输入输出数据记录在表 6.4 中。

绝对值电路根据式 (3.2) 实现：先通过比较器判断 **aIn** 与 **bIn** 的大小，利用数据选择器选出最大值 **Max** 和最小值 **Min**，再将最大值与最小值的补码（取反加一）相加，得到两数之差的绝对值。

从仿真结果可以看出，电路能够正确计算两个 4 位无符号数之差的绝对值：

- 当 **aIn**=3, **bIn**=12 时, $|3 - 12| = 9$, 输出 **out**=9, 与预期一致。
- 当 **aIn**=10, **bIn**=4 时, $|10 - 4| = 6$, 输出 **out**=6, 结果正确。
- 当 **aIn**=5, **bIn**=5 时, 两数相等, 差值为 0, 输出 **out**=0。
- 当 **aIn**=12, **bIn**=0 时, $|12 - 0| = 12$, 输出 **out**=12, 符合预期。

所有测试用例均验证了电路的正确性，表明通过调用比较器、数据选择器和全加器构建的绝对值电路能够可靠地完成两数之差的绝对值计算。该设计体现了“自顶而下”的模块化设计思想，各子模块可独立验证和复用。

6.5 模式比较器仿真测试

	Msgs						
+ /ModeComparator_tb/out	8'd132	122	167	68	5	103	132
+ /ModeComparator_tb/a	8'd132	33	167	68	5	112	132
+ /ModeComparator_tb/b	8'd141	122	4	68	5	103	141
+ /ModeComparator_tb/m	1'd1	0			1		

图 6.6: 模式比较器电路 ModelSim 波形仿真

```

VSIM 27> run -all
# MAX( 33 , 122 )= 122
# MAX( 167 , 4 )= 167
# MAX( 68 , 68 )= 68
# MIN( 5 , 5 )= 5
# MIN( 112 , 103 )= 103
# MIN( 132 , 141 )= 132
# ** Note: $stop : C:/Users/57195/Desktop/SBZJU/DigitalSystemDesignEx/lab5/src/ModeComparator_tb.v(27)
# Time: 120 ps Iteration: 0 Instance: /ModeComparator_tb

```

图 6.7: 模式比较器电路 ModelSim 控制台输出

a (十进制)	b (十进制)	m (模式选择)	out (输出)
33	122	0	122
167	4	0	167
68	68	0	68
5	5	1	5
112	103	1	103
132	141	1	132

表 6.5: 模式比较器电路 ModelSim 仿真波形结果

为验证模式比较器电路的功能正确性, 使用 ModelSim 对 8 位模式比较器 (参数 $n=8$) 进行了功能仿真。仿真波形如图 6.6 所示, 控制台输出如图 6.7 所示, 对应的输入输出数据记录在表 6.5 中。

模式比较器电路的核心思想是: 先通过比较器模块 **comp** 判断输入 a 与 b 的大小关系, 得到标志信号 **agb** ($a > b$); 然后根据 **agb** 确定最大值 **max_out** 和最小值 **min_out**; 最后依据模式控制信号 **m** 选择输出: 当 $m=0$ 时输出最大值, 当 $m=1$ 时输出最小值。

从仿真结果可以看出, 电路在所有测试用例下均能正确输出:

- 当 $m=0$ 时, 输出应为 $\text{MAX}(a, b)$ 。例如表中第一组 ($a = 33, b = 122$), 最大值为 122, 输出 **out**=122; 第二组 ($a = 167, b = 4$), 最大值为 167, 输出 167; 第三组 ($a = 68, b = 68$), 两数相等, 最大值即为 68, 输出 68。
- 当 $m=1$ 时, 输出应为 $\text{MIN}(a, b)$ 。例如第四组 ($a = 5, b = 5$), 最小值为 5, 输出 5; 第五组 ($a = 112, b = 103$), 最小值为 103, 输出 103; 第六组 ($a = 132, b = 141$), 最小值为 132, 输出 132。

出 132。

所有测试用例均与预期结果完全一致，证明模式比较器电路的功能正确。该设计通过复用已实现的比较器模块，仅用少量组合逻辑便实现了最大值/最小值的模式可选输出，体现了“自顶而下”模块化设计的优势：各子模块（比较器）可独立验证，顶层电路结构清晰，易于维护和扩展。

七、 综合测试

7.1 Vivado 综合与约束

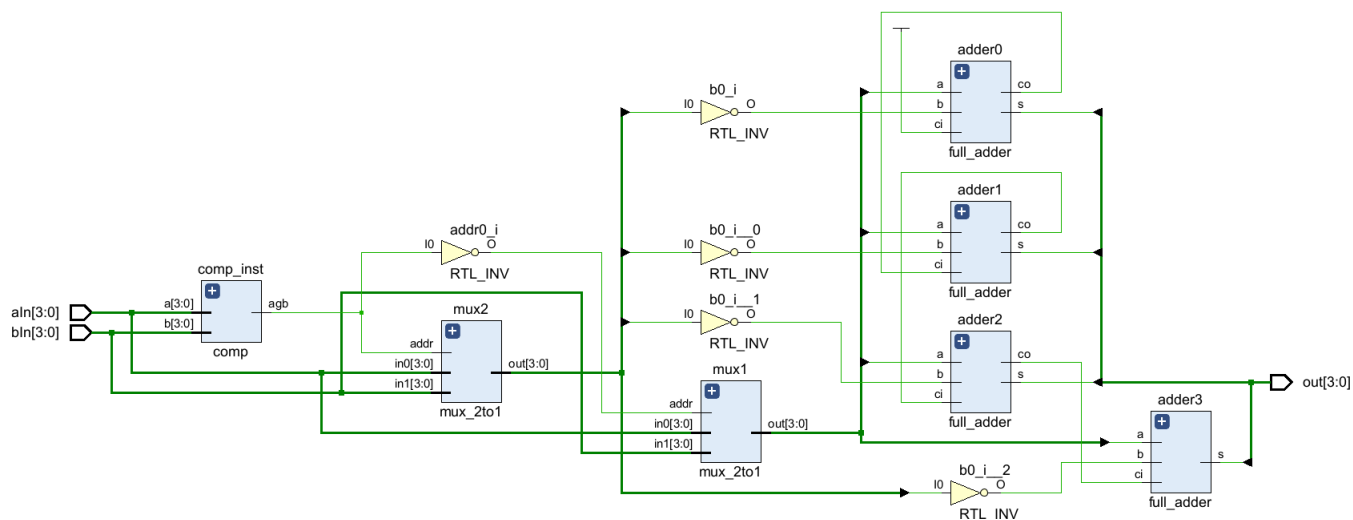


图 7.1: Vivado 综合原理图

从综合原理图中可以看到，其描述的层次化结构与我们设计理念一致，即先进行比较，然后通过选择器进行数据选取，最后利用全加器进行求和，得到两数之差的绝对值。

下面的约束文件展示了引脚的约束：

```
1 set_property IOSTANDARD LVCMOS33 [get_ports {aIn[3]}]
2 set_property IOSTANDARD LVCMOS33 [get_ports {aIn[2]}]
3 set_property IOSTANDARD LVCMOS33 [get_ports {aIn[1]}]
4 set_property IOSTANDARD LVCMOS33 [get_ports {aIn[0]}]
5 set_property IOSTANDARD LVCMOS33 [get_ports {bIn[3]}]
6 set_property IOSTANDARD LVCMOS33 [get_ports {bIn[2]}]
7 set_property IOSTANDARD LVCMOS33 [get_ports {bIn[1]}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {bIn[0]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {out[3]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {out[2]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {out[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {out[0]}]
13 set_property PACKAGE_PIN G22 [get_ports {aIn[3]}]
```

```
14 set_property PACKAGE_PIN G21 [get_ports {aIn[2]}]
15 set_property PACKAGE_PIN F21 [get_ports {aIn[1]}]
16 set_property PACKAGE_PIN E22 [get_ports {aIn[0]}]
17 set_property PACKAGE_PIN M17 [get_ports {bIn[3]}]
18 set_property PACKAGE_PIN K13 [get_ports {bIn[2]}]
19 set_property PACKAGE_PIN J16 [get_ports {bIn[1]}]
20 set_property PACKAGE_PIN H17 [get_ports {bIn[0]}]
21 set_property PACKAGE_PIN U16 [get_ports {out[3]}]
22 set_property PACKAGE_PIN T16 [get_ports {out[2]}]
23 set_property PACKAGE_PIN T15 [get_ports {out[1]}]
24 set_property PACKAGE_PIN T14 [get_ports {out[0]}]
```

I/O 名称	I/O	引脚编号	接口类型
aIn[0]	Input	E22	LVCMOS33
aIn[1]	Input	F21	LVCMOS33
aIn[2]	Input	G21	LVCMOS33
aIn[3]	Input	G22	LVCMOS33
bIn[0]	Input	H17	LVCMOS33
bIn[1]	Input	J16	LVCMOS33
bIn[2]	Input	K13	LVCMOS33
bIn[3]	Input	M17	LVCMOS33
out[0]	Output	T14	LVCMOS33
out[1]	Output	T15	LVCMOS33
out[2]	Output	T16	LVCMOS33
out[3]	Output	T16	LVCMOS33

表 7.1: 引脚约束内容

根据内部逻辑，当拨动开关向上和 LED 亮起时代表逻辑 1，否则代表逻辑 0；拨动开关左边四个分别表示 b[3]，b[2]，b[1]，b[0]，右边四个分别表示 a[3]，a[2]，a[1]，a[0]，而上方 LED 表示输出，右边四个 LED 分别表示 out[3]，out[2]，out[1]，out[0]。

7.2 下板测试

在 Vivado 里完成引脚约束、生成比特流等步骤后，将代码下放到开发板并进行测试。



图 7.2: $a > b$ 的场景

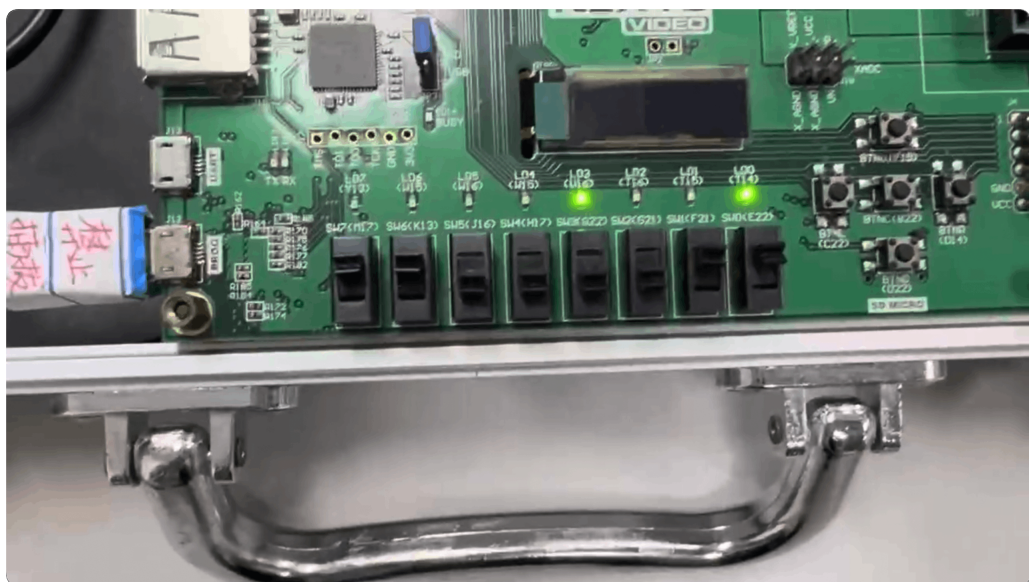


图 7.3: $b > a$ 的场景

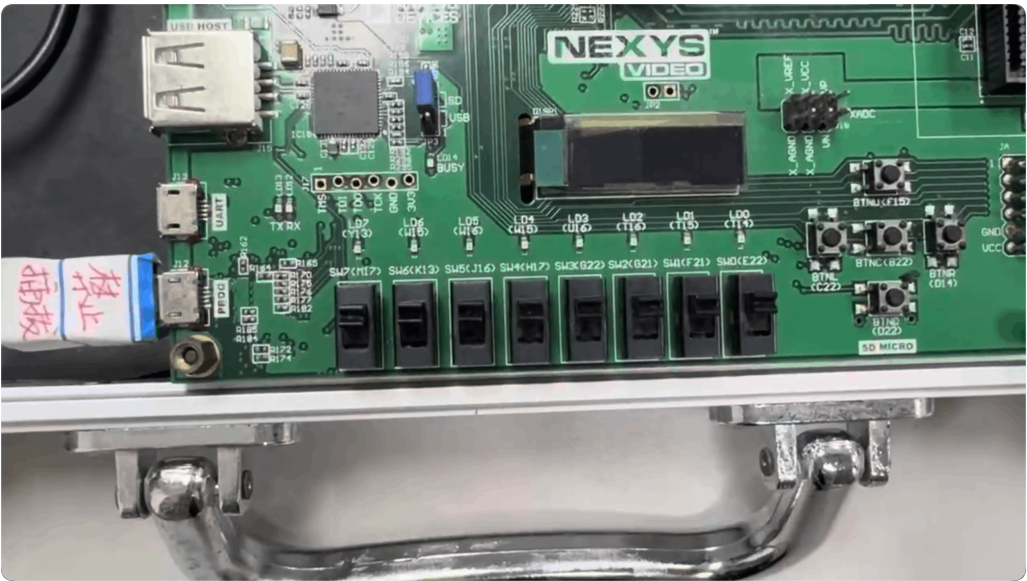


图 7.4: a=b 的场景

<i>a</i> (十进制)	<i>b</i> (十进制)	<i>out</i> (输出, $ a - b $)
11	9	2
3	12	9
15	15	0

表 7.2: 两数之差的绝对值电路下板测试

为验证设计在实际硬件上的正确性，将综合生成的比特流下载至开发板进行测试。输入 **aIn** 和 **bIn** 通过板载拨动开关设置（开关向上表示逻辑 1），输出 **out** 通过 LED 显示（LED 亮表示逻辑 1）。选取三组典型测试用例，结果如表 7.2 所示。

- 当 **aIn**=11（二进制 1011），**bIn**=9（二进制 1001）时，两数之差绝对值为 2，输出 LED 显示 0010（十进制 2），与预期一致。
- 当 **aIn**=3（0011），**bIn**=12（1100）时， $|3 - 12| = 9$ ，输出 LED 显示 1001（十进制 9），结果正确。
- 当 **aIn**=15（1111），**bIn**=15（1111）时，两数相等，差值为 0，所有 LED 熄灭，输出 0。

下板测试结果与 ModelSim 功能仿真结果完全吻合，证明了绝对值之差电路在实际 FPGA 器件上能够正常工作，同时也验证了引脚约束和综合实现的正确性。

八、心得与体会

8.1 实验结论

1. 成功实现了 4 位无符号数绝对值之差电路，仿真与下板测试均证明其能正确计算 $|a - b|$ 。

- 成功实现了 8 位模式比较器电路, 可根据模式信号 m 输出两数的最大值或最小值。
- 通过多层次结构设计, 验证了“自顶而下”设计方法的可行性: 顶层电路通过调用已验证的子模块(比较器、数据选择器、全加器)快速完成复杂功能, 各模块可独立仿真、综合, 提高了设计效率和可靠性。

8.2 问题与解决方案

- 数据比较器时出现有时可以正确比较大小, 但有时又无法比较大小的问题。

原因: 在最初代码中, 只有 `if (a[i] > b[i]) begin` 判断语句而缺少后续判断, 导致部分数字在比较大小时, b 的前几位已经比 a 大, 但程序没有及时终止, 而是继续比较下去, 直到终止(默认 $a = b$), 从而输出错误结论。

解决: 添加后续 `else if (a[i] < b[i]) begin` 判断语句, 使得可以在 b 比 a 大时及时退出循环。

- 进行 Vivado 综合时打开 Schematic 出现报错问题。

原因: 部分源文件的 `parameter` 未设置默认参数, 虽然不影响功能使用, 但是在进行 Vivado 综合时仍会出现报错提示。

解决: 将 `parameter` 参数默认设置为 8, 可以通过调用模块时的参数传递改变默认值。

8.3 感悟与体会

通过本次实验, 我不仅巩固了 Verilog 语法和组合逻辑设计方法, 还体会到了模块化设计对复杂数字系统开发的重要性。在遇到问题时, 通过分析仿真波形、查阅资料和反复调试, 最终找到解决方案, 这对我今后的工程实践具有宝贵的参考价值。

九、 思考题

- 求两数之差的绝对值电路, 若输入为有符号数(补码), 该怎样设计? 请画出原理框图并作简要说明。

设计说明: 若输入为 n 位的有符号数(补码表示), 直接比较大小或求差可能会因为溢出而导致结果错误。因此, 为了防止计算差值时发生溢出, 首先需要将输入数据 A 和 B 进行符号位扩展, 扩充为 $n + 1$ 位。

扩展后, 利用减法器(或采用加法器加上补码的方式: $A + (\sim B) + 1$) 计算两数之差 $D = A_{ext} - B_{ext}$ 。由于 D 是有符号数, 其最高位(符号位)可以直接反映差值的正负:

- 当 D 的符号位为 0 时(即 $D \geq 0$), 绝对值就是差值本身, 数据选择器直接输出 D 。
- 当 D 的符号位为 1 时(即 $D < 0$), 绝对值为差值的相反数, 需要对 D 求补(即取反加一), 数据选择器输出 $\sim D + 1$ 。

原理框图如图 9.1 所示。

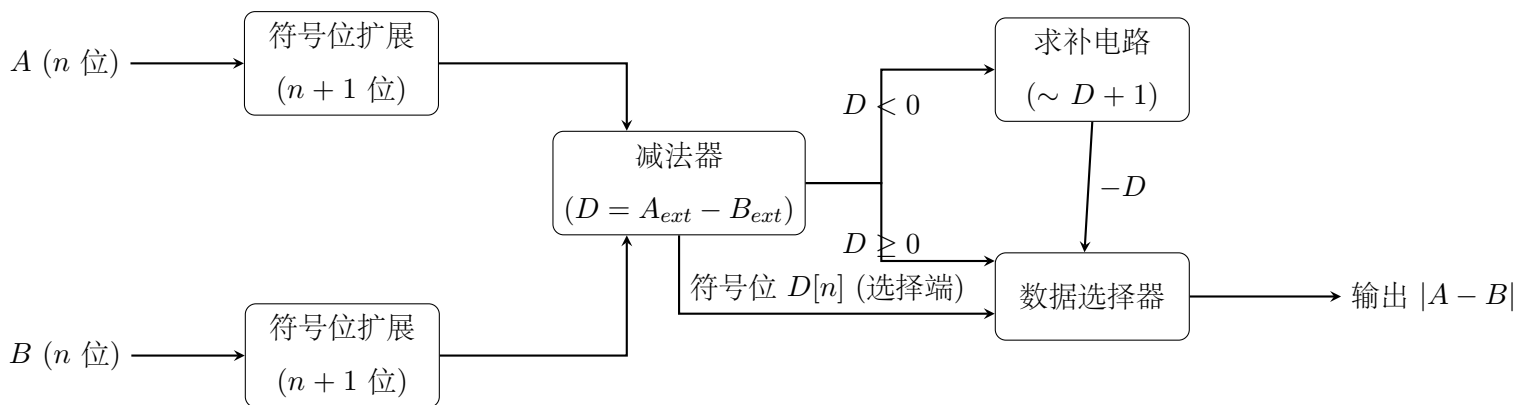


图 9.1: 有符号数绝对值之差电路原理框图

2. 能否用加法器实现比较器，若能，请画出原理框图。

设计说明: 能用加法器实现比较器。比较两个无符号数 A 和 B 的大小，可以通过计算差值 $A - B$ 来实现，而减法可以转化为补码加法： $A - B = A + (\sim B) + 1$ 。

使用一个带有进位输入 (C_{in}) 的 n 位加法器，将 A 直接接入， B 按位取反后接入，并将最低位进位 C_{in} 置为 1。根据加法器的输出和 S 及进位输出 C_{out} 即可判断大小：

- 等于 (**aeb**): 若 $A = B$ ，则 $A - B = 0$ ，即加法器的和输出 S 的所有位均为 0。可以通过对 S 的所有位进行一次 NOR (或非) 运算得出。
- 大于 (**agb**): 对于无符号数，若 $A \geq B$ ，则加法过程会产生进位，即 $C_{out} = 1$ 。为了排除 $A = B$ 的情况，需要满足 $C_{out} == 1$ 且 $S \neq 0$ 。即 **agb** = $C_{out} \wedge (\sim \text{aeb})$ 。
- 小于 (**alb**): 若 $A < B$ ，则加法过程不会产生进位，即 $C_{out} = 0$ 。因此 **alb** = $\sim C_{out}$ 。

原理框图如图 9.2 所示。

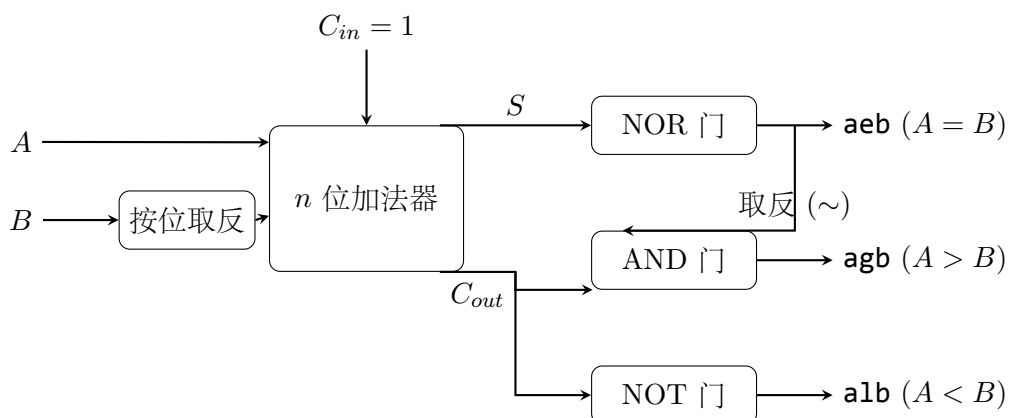


图 9.2: 用加法器实现比较器原理框图

3. 调用模块时怎样进行参数传递？若模块有多个参数，模块实例的格式如何？

解答: 在 Verilog HDL 中，在调用带有参数的模块（如带有 **parameter** 的模块）时，通常使用 **#()** 符号来进行参数传递。有两种常见的传参方式：按位置传递和按名称传递（推荐）。

若模块有多个参数，推荐使用**按名称（具名）映射**的方法进行传递，这样可以避免参数顺序混淆。

多个参数之间使用逗号(,) 隔开。模块实例的通用格式如下:

```
1 // 被调用的模块声明示例
2 // module my_module #(parameter PARAM1 = 8, parameter PARAM2 = 1) (...)
3
4 // 调用并传递多个参数的格式:
5 模块名 #(.参数名1(传递值1), .参数名2(传递值2)) 实例名 (
6     .端口1(连接的信号1),
7     .端口2(连接的信号2),
8     ...
9 );
10
11 // 具体举例:
12 my_module #(.PARAM1(16), .PARAM2(0)) u_my_module (
13     .a(wire_a),
14     .b(wire_b)
15 );
```

使用这种格式, 不仅提高了代码的可读性, 即使底层模块的参数定义顺序发生改变, 上层调用逻辑也不受影响。